Implementación de Regresión Logística y Algoritmos de Clasificación de Machine Learning Supervisado con Scikit-Learn

Jerson Baquero Cháves
Universidad de Cundinamarca
Informe individual de Machine Learning Supervisado
Octubre de 2026

Repositorio en GitHub: https://github.com/Jerson09/R1-A2-S8_ml_supervisado

Resumen

Este informe presenta la implementación, en Google Colab, de la Regresión Logística y de cinco algoritmos de clasificación supervisada (Decision Tree, Random Forest, Support Vector Machine, K Nearest Neighbor y Naive Bayes) mediante las librerías Scikit-Learn, Pandas, NumPy y Matplotlib. Todos los modelos se entrenaron sobre el conjunto de datos *Breast Cancer Wisconsin (Diagnostic)*, compuesto por 569 casos y 30 variables, con una partición estratificada de 75 % para entrenamiento y 25 % para prueba. El desempeño se evaluó con exactitud, precisión, *recall*, F1, área bajo la curva ROC (AUC), matriz de confusión y validación cruzada de cinco pliegues. La Regresión Logística obtuvo la mayor exactitud en la partición de prueba (98,6 %) y el mayor AUC (0,998), seguida por SVM y KNN (97,9 %). Se discuten las diferencias entre modelos, el efecto del umbral de decisión y las limitaciones del experimento.

Palabras clave: aprendizaje supervisado, regresión logística, clasificación, Scikit-Learn, matriz de confusión

Implementación de Regresión Logística y Algoritmos de Clasificación de Machine Learning Supervisado con Scikit-Learn

Introducción

El aprendizaje supervisado es la rama del *machine learning* en la que un algoritmo aprende a partir de ejemplos etiquetados, es decir, pares de datos de entrada y la salida esperada, con el fin de predecir la etiqueta de observaciones nuevas (IBM, s. f.). Cuando la etiqueta es una categoría, el problema se denomina clasificación; cuando es un valor numérico continuo, regresión (Hastie et al., 2009).

El objetivo de este informe es implementar la Regresión Logística y los principales algoritmos de clasificación (Decision Tree, Random Forest, SVM, KNN y Naive Bayes), analizar sus resultados con métricas de clasificación y comparar su comportamiento sobre un mismo problema. Los notebooks desarrollados en Google Colab se encuentran en el repositorio de GitHub indicado en la sección de Repositorio.

Marco conceptual

Machine Learning Supervisado

En el aprendizaje supervisado se dispone de un conjunto de entrenamiento con variables predictoras y una variable objetivo conocida. El modelo ajusta sus parámetros para minimizar el error entre sus predicciones y las etiquetas reales, y luego se evalúa con datos que no vio durante el entrenamiento para estimar su capacidad de generalización (Hastie et al., 2009). Por esa razón, el flujo de trabajo habitual incluye separar los datos en subconjuntos de entrenamiento y prueba, y complementar la evaluación con validación cruzada.

Regresión Logística

La Regresión Logística es un modelo lineal para clasificación que estima la probabilidad de pertenencia a una clase mediante la función sigmoide aplicada a una combinación lineal de las variables: $P(y = 1 \mid x) = 1 / (1 + e^{-z})$, con $z = b_0 + b_1x_1 + ... + b_nx_n$ (Hosmer et al., 2013). La observación se asigna a la clase positiva cuando la probabilidad supera un umbral, generalmente 0,5. Sus coeficientes son interpretables, pues cada uno indica cómo cambia el logaritmo de las probabilidades (*log-odds*) de la clase positiva ante un cambio unitario en la variable correspondiente.

Decision Tree

Un árbol de decisión divide recursivamente el espacio de variables mediante preguntas sobre una variable a la vez, eligiendo en cada nodo la división que más reduce la impureza, por ejemplo, el índice de Gini o la entropía (Quinlan, 1986). Es fácil de interpretar, pero tiende al sobreajuste si no se limita su profundidad.

Random Forest

Random Forest combina muchos árboles entrenados con muestras aleatorias de los datos y subconjuntos aleatorios de variables; la predicción final surge de la votación de los árboles (Breiman, 2001). Esta estrategia reduce la varianza respecto de un único árbol y ofrece una medida de importancia de variables.

Support Vector Machine (SVM)

SVM busca el hiperplano que separa las clases con el mayor margen posible; mediante funciones núcleo (*kernels*), como el RBF, puede construir fronteras no lineales (Cortes y Vapnik, 1995). Es sensible a la escala de las variables, por lo que requiere estandarización.

K Nearest Neighbor (KNN)

KNN clasifica una observación según la clase mayoritaria entre sus *k* vecinos más cercanos en el espacio de variables (Cover y Hart, 1967). No construye un modelo explícito durante el entrenamiento y depende de la métrica de distancia y del valor de *k*.

Naive Bayes

Naive Bayes aplica el teorema de Bayes suponiendo independencia condicional entre las variables dadas la clase. Aunque este supuesto rara vez se cumple, el clasificador suele funcionar bien en la práctica (Domingos y Pazzani, 1997). En este trabajo se usó la versión gaussiana, adecuada para variables continuas.

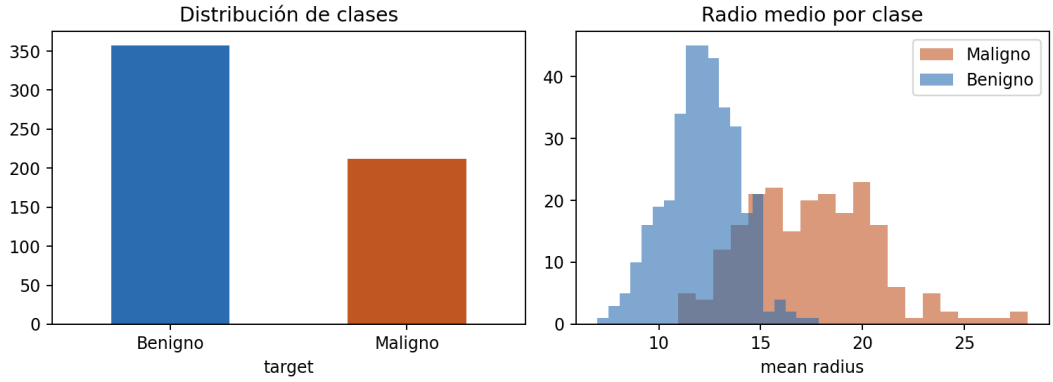
Metodología

Entorno y datos

Los experimentos se desarrollaron en Python con Scikit-Learn (Pedregosa et al., 2011), Pandas (McKinney, 2010), NumPy y Matplotlib (Hunter, 2007), en dos notebooks de Google Colab: uno dedicado a la Regresión Logística y otro a los demás clasificadores. Se utilizó el conjunto *Breast Cancer Wisconsin (Diagnostic)*, derivado de imágenes digitalizadas de aspiraciones con aguja fina de masas mamarias (Street et al., 1993), disponible en Scikit-Learn. Contiene 569 observaciones y 30 variables numéricas que describen los núcleos celulares; 212 casos son malignos y 357 benignos (Figura 1). En el código, la clase benigna se codifica como 1 y se toma como clase positiva.

Figura 1

Distribución de clases y del radio medio por clase



Preparación y evaluación

Los datos se dividieron de forma estratificada en 75 % para entrenamiento (426 casos) y 25 % para prueba (143 casos), con semilla aleatoria fija (*random_state* = 42) para garantizar la reproducibilidad. En la Regresión Logística, SVM y KNN se aplicó estandarización (*StandardScaler*) dentro de un *pipeline*, de modo que la escala se aprende solo con los datos de entrenamiento. Los hiperparámetros principales fueron: árbol con profundidad máxima de 4; bosque con 200 árboles; SVM con núcleo RBF; KNN con k = 5; y Naive Bayes gaussiano. No se realizó una búsqueda sistemática de hiperparámetros.

Las métricas se calcularon sobre el conjunto de prueba tomando la clase benigna como positiva: exactitud (proporción de aciertos), precisión (de los casos predichos como benignos,

cuántos lo son), *recall* (de los casos realmente benignos, cuántos se detectan), F1 (media armónica de precisión y *recall*) y AUC. Además se reportó la exactitud media de una validación cruzada estratificada de cinco pliegues sobre los 569 casos.

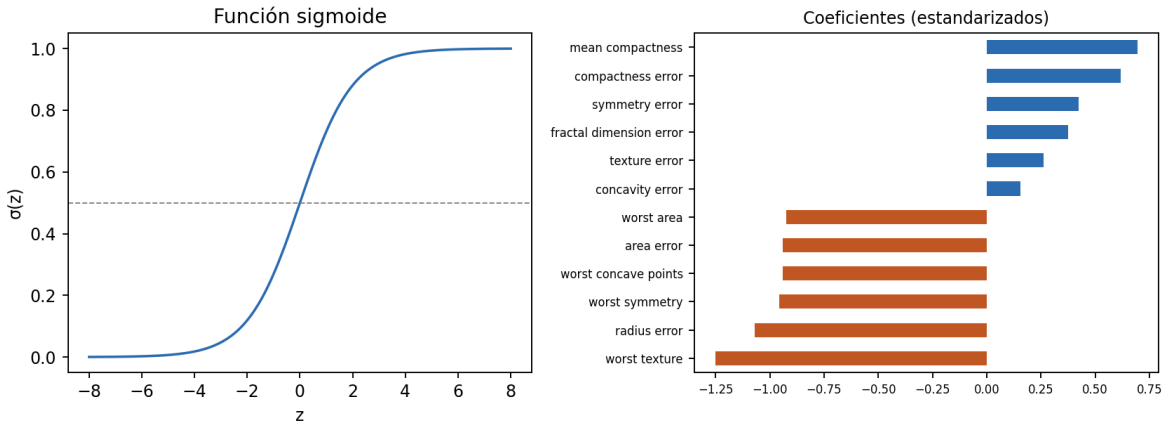
Resultados

Regresión Logística

La Regresión Logística alcanzó una exactitud de 98.6 % en la partición de prueba, con precisión de 0.989, *recall* de 0.989, F1 de 0.989 y AUC de 0.998. Su matriz de confusión muestra 52 malignos y 89 benignos clasificados correctamente, con un caso benigno clasificado como maligno y un caso maligno clasificado como benigno. La Figura 2 presenta la función sigmoide y los coeficientes estandarizados de mayor magnitud: las variables *worst texture*, *radius error* y *worst concave points* presentan coeficientes negativos, es decir, valores altos de estas variables reducen la probabilidad de que el tumor sea benigno.

Figura 2

Función sigmoide y coeficientes estandarizados de la Regresión Logística



Se evaluó además el efecto del umbral de decisión (Tabla 1). Al aumentar el umbral, el modelo exige más certeza para declarar un caso como benigno: la precisión sube, pero el *recall* baja y aumentan los benignos clasificados como malignos. Con un umbral de 0,3 se obtuvo un *recall* de 1,000 a costa de tres malignos clasificados como benignos; con 0,9 no hubo malignos clasificados como benignos, pero 11 benignos fueron descartados como malignos.

Tabla 1

Efecto del umbral de decisión en la Regresión Logística

Umbral	Precisión	Recall	Malignos clasificados como benignos	Benignos clasificados como malignos
0.3	0.968	1.000	3	0
0.5	0.989	0.989	1	1
0.7	0.988	0.933	1	6
0.9	1.000	0.878	0	11

Nota. Clase positiva: benigno. Conjunto de prueba, n = 143.

Comparación de los algoritmos de clasificación

La Tabla 2 resume el desempeño de los seis modelos. La Regresión Logística obtuvo la mayor exactitud y el mayor AUC en la partición de prueba; SVM y KNN alcanzaron 97,9 %. KNN logró un *recall* perfecto (no dejó escapar ningún benigno), aunque con una precisión menor. Random Forest, Decision Tree y Naive Bayes quedaron entre 93,7 % y 95,8 %. La validación cruzada confirma un orden similar: SVM (0,977) y Regresión Logística (0,974) lideran, mientras que Decision Tree (0,923) y Naive Bayes (0,939) quedan por debajo.

Tabla 2

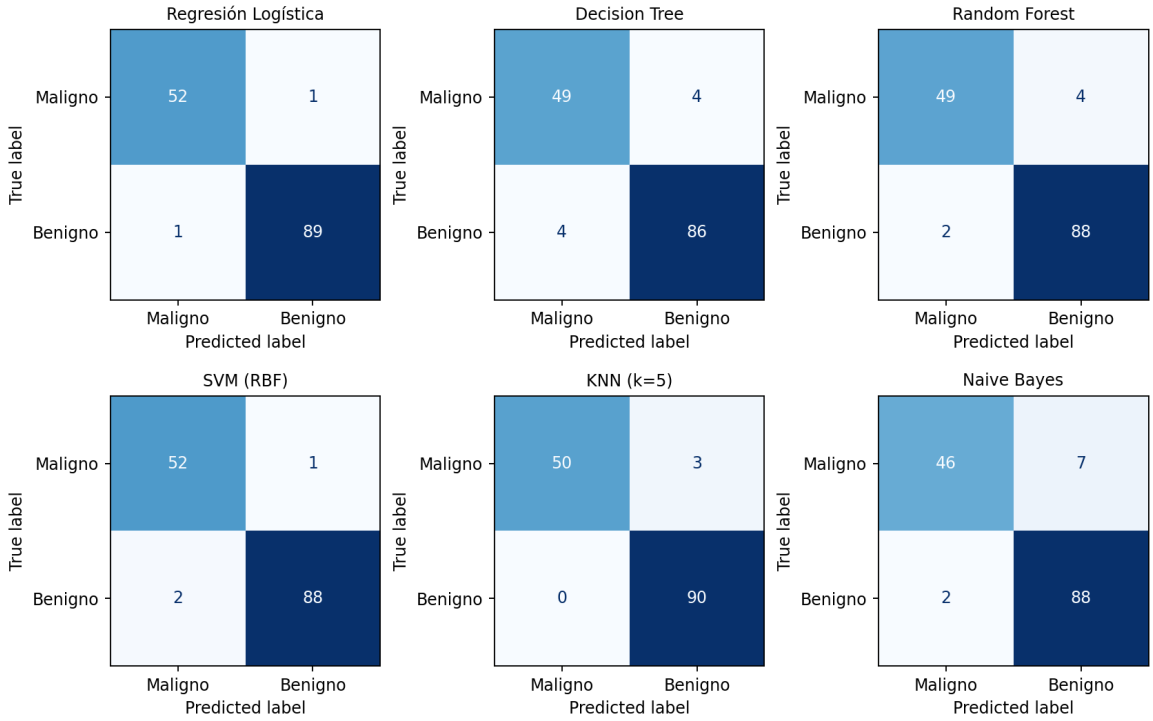
Métricas de los modelos en el conjunto de prueba y en validación cruzada

Modelo	Exactitud	Precisión	Recall	F1	AUC	CV (media ± d. e.)
Regresión Logística	0.986	0.989	0.989	0.989	0.998	0.974 ± 0.017
Decision Tree	0.944	0.956	0.956	0.956	0.938	0.923 ± 0.029
Random Forest	0.958	0.957	0.978	0.967	0.995	0.954 ± 0.010
SVM (RBF)	0.979	0.989	0.978	0.983	0.997	0.977 ± 0.016
KNN (k=5)	0.979	0.968	1.000	0.984	0.985	0.963 ± 0.018
Naive Bayes	0.937	0.926	0.978	0.951	0.989	0.939 ± 0.024

Nota. Clase positiva: benigno. CV = exactitud en validación cruzada estratificada de 5 pliegues (n = 569).

Figura 3

Matrices de confusión de los seis modelos en el conjunto de prueba



Nota. Filas: clase real; columnas: clase predicha.

Tabla 3

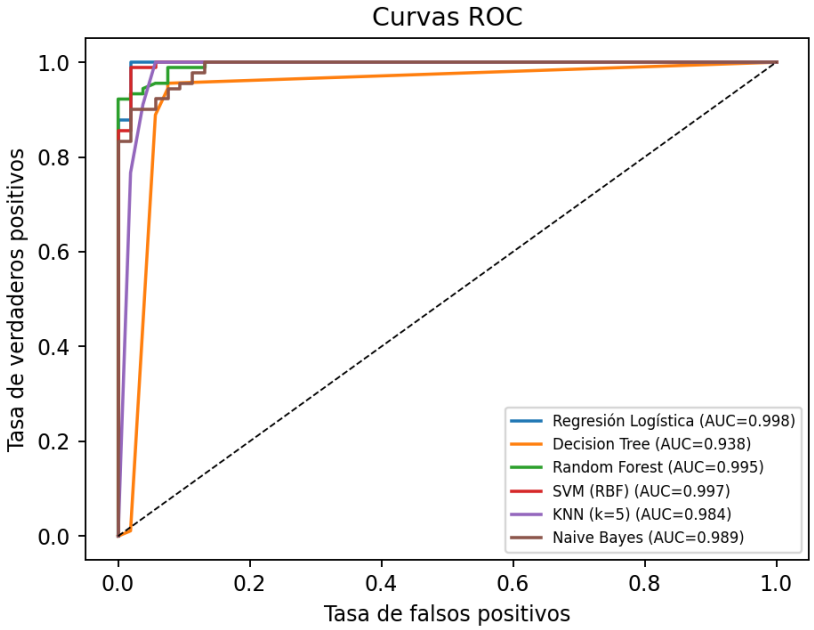
Errores de clasificación por modelo en el conjunto de prueba (n = 143)

Modelo	Malignos clasificados como benignos	Benignos clasificados como malignos	Errores totales
Regresión Logística	1	1	2
Decision Tree	4	4	8
Random Forest	4	2	6
SVM (RBF)	1	2	3
KNN (k=5)	3	0	3
Naive Bayes	7	2	9

La Tabla 3 permite distinguir el tipo de error. Desde un punto de vista clínico, clasificar un tumor maligno como benigno es el error más costoso, porque podría retrasar un tratamiento. Por eso, además de la exactitud global, conviene vigilar este tipo de error y considerar ajustar el umbral, como se mostró en la Tabla 1.

Figura 4

Curvas ROC de los modelos en el conjunto de prueba

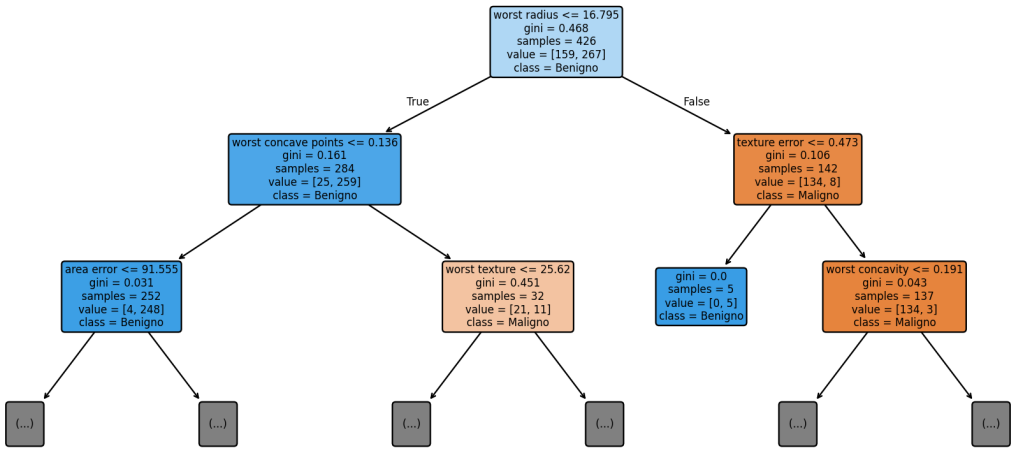


Resultados por algoritmo

Decision Tree. Con profundidad máxima de 4 obtuvo 94,4 % de exactitud y fue el modelo con menor AUC (0,938), lo cual es coherente con que un solo árbol produce probabilidades poco finas. Su ventaja es la interpretabilidad (Figura 5).

Figura 5

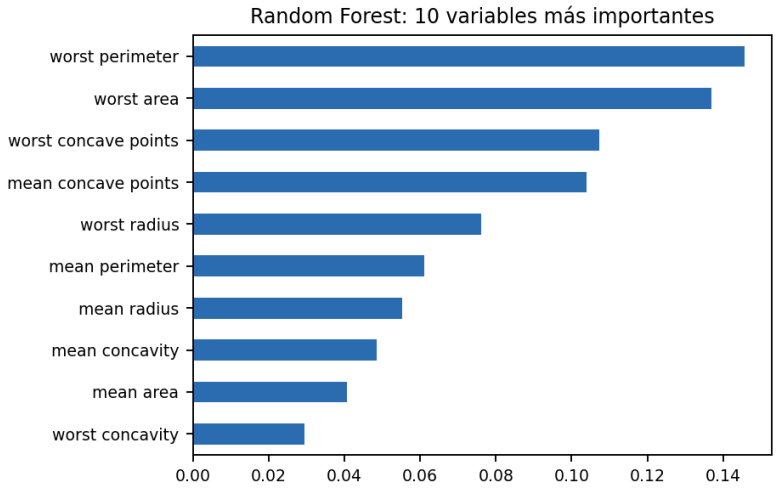
Primeros niveles del árbol de decisión



Random Forest. Alcanzó 95,8 % de exactitud y un AUC de 0,995. Mejoró al árbol individual en todas las métricas globales y fue el modelo más estable en validación cruzada (desviación estándar de 0,010). Las variables más importantes fueron *worst perimeter*, *worst area* y *worst concave points* (Figura 6).

Figura 6

Importancia de variables en Random Forest

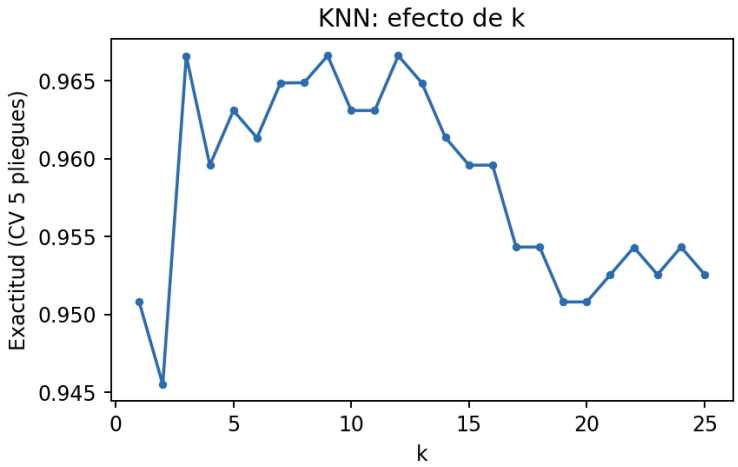


SVM. Con núcleo RBF obtuvo 97,9 % de exactitud, precisión de 0,989 y la mayor exactitud en validación cruzada (0,977).

KNN. Con k = 5 logró 97,9 % de exactitud y *recall* de 1,000. La exploración de k mediante validación cruzada (Figura 7) indicó que el mejor valor fue k = 9, con una exactitud media de 0,967, mayor que la de k = 5 (0,963). La diferencia es pequeña, por lo que no cambia las conclusiones.

Figura 7

Exactitud de KNN en validación cruzada según el valor de k



Naive Bayes. Obtuvo 93,7 % de exactitud, el valor más bajo del conjunto, aunque con un AUC de 0,989. Esto sugiere que ordena bien los casos por probabilidad, pero su umbral por defecto no es el óptimo, probablemente porque las variables están muy correlacionadas y violan el supuesto de independencia.

Discusión

El buen desempeño de la Regresión Logística y de SVM indica que las clases de este conjunto son casi linealmente separables una vez estandarizadas las variables, de modo que modelos más complejos no aportan una ventaja clara. Random Forest, a pesar de su fama de robustez, quedó por debajo de los modelos lineales en la partición de prueba, pero esta diferencia equivale a pocos casos: con 143 observaciones de prueba, un caso adicional mal clasificado modifica la exactitud en aproximadamente 0,7 puntos porcentuales. Por eso las diferencias de uno o dos casos entre modelos no deben interpretarse como superioridad definitiva.

Entre las limitaciones del trabajo se cuentan: el uso de una única partición de prueba, la ausencia de optimización de hiperparámetros (por ejemplo, con *GridSearchCV*), y el uso de un conjunto de datos pequeño y relativamente sencillo. Los resultados, por tanto, ilustran el uso de los algoritmos y de las métricas, pero no deben extrapolarse a otros problemas.

Conclusiones

- Se implementaron en Google Colab la Regresión Logística y cinco algoritmos de clasificación supervisada sobre un mismo conjunto de datos, con un procedimiento reproducible de partición, estandarización y evaluación.
- La Regresión Logística obtuvo el mejor desempeño en la partición de prueba (exactitud de 98,6 % y AUC de 0,998), y SVM fue el mejor en validación cruzada (0,977); ambos modelos fueron los más consistentes.
- La exactitud global no basta: la matriz de confusión, la precisión y el *recall* permiten identificar qué tipo de error comete cada modelo, y el umbral de decisión permite equilibrarlos según el costo de cada error.
- Como trabajo futuro se recomienda ajustar hiperparámetros, repetir la validación con distintas particiones y probar conjuntos de datos más grandes y desbalanceados.

Repositorio

Los notebooks desarrollados (*01_regresion_logistica.ipynb* y *02_clasificadores.ipynb*) están disponibles en el siguiente repositorio de GitHub:
https://github.com/Jerson09/R1-A2-S8_ml_supervisado

Referencias

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*(3), 273–297. https://doi.org/10.1007/BF00994018

Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory, 13*(1), 21–27. https://doi.org/10.1109/TIT.1967.1053964

Domingos, P., & Pazzani, M. (1997). On the optimality of the simple Bayesian classifier under zero-one loss. *Machine Learning, 29*(2–3), 103–130. https://doi.org/10.1023/A:1007413511361

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2.ª ed.). Springer.

Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression* (3.ª ed.). Wiley.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

IBM. (s. f.). *What is supervised learning?* IBM Think. https://www.ibm.com/think/topics/supervised-learning

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning, 1*(1), 81–106. https://doi.org/10.1007/BF00116251

Street, W. N., Wolberg, W. H., & Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. *Proceedings of SPIE, 1905*, 861–870. https://doi.org/10.1117/12.148698